STM32IDE, MX 와 SPI, UART를 이용한 디스플레이 소품 제작

핵심 요약 키워드

: STM32, C, SPI, UART

: STM32F103RB, Nucleo-F103RB, W25Q64, ILI9341

: 부품조합, 최소 핀 사용, 디버깅

프로젝트 개요

- 개발 기간 : 2026.06.17 ~ 2026.11 (6달)
- 목표 : IDLE 상태에서 ILI9341에 움직이는 사진을 출력하고,
  터치할 때에는 다른 움직이는 사진을 ILI9341에 출력한다.
- 사용 MCU/보드 : STM32F103RB / Nucleo-F103RB
- 본인 담당 : 개인 프로젝트 (= 가설수립, 문제 해결방안 단계 나누기, 단계별로 수행)

결과물

- 구동 영상 : https://youtu.be/cVpCpbx06ak

프로젝트의 연결 구상도



마주했던 큰 문제

1. STM32F103RB 자체의 메모리 부족

ILI9341은 320 * 240 픽셀을 쓰고, 픽셀은 RBG565 값을 쓰기에 픽셀 당 2byte를 필요로 합니다.
즉 대략 사진 1장 당 150KB를 필요로 합니다.
또한 사진을 대략 8~9장 담아야 하기에 1200KB (1Mb 이상)을 필요로 합니다

하지만 STM32F103의 flash 메모리는 128 KB로, 정수 상수로 사진 1장을 담기도 불가능한 크기였습니다.
따라서 코드 내에 문자열 상수로 1Mb 의 사진을 담는 것은 가능한 방법이 아닙니다.

그래서 부품을 알아보던 중, 메모리의 기능을 담당할 수 있는 W25Q64 부품을 찾았습니다.
W25Q64를 보다 효율적으로 이용하기 위해 터미널 이용법을 익히고나니,
다음 작동 구조가 완성되었습니다

a. 업로드 단계

PC 터미널 - [UART2] - MCU - [SPI1] - W25Q64
{입력} { HEX 문자열 정수 변환 } { 정수 보관 }

b. ILI9341로 다운로드 단계

W25Q64 - [SPI1] - MCU - [SPI1] - ILI9341
{정수 출력} { CS 핀 관리, 정수 일시보관 } { 정수 입력받음, 디스플레이 출력 }



2. SPI 의 장점 극대화

SPI는 1개의 마스터와 여러개의 슬레이브 장치가 연결되는 것을 허용합니다.
이 점을 극대화 하고자, W25QXX, ILI9341 둘 다 통신 장치로 SPI1을 활용했습니다.

두 장치 다 MOSI, MISO, CLK 는 Nucleo 보드 내에 같은 헤더핀을 사용하지만
CS, DC 핀은 각각 별개의 핀을 사용함으로써, Nucleo 보드 내에 3개의 핀,
그리고 SPI 장치를 단 1개만 씀으로써 실제로 요구되는 회로의 크기를 줄일 수 있었습니다.

3. 불완전한 문자열 업로드 시 merge 기능 제작

빠른 통신속도의 UART를 다루고 있다보니, 간혹 문자열이 잘려서 입력되는 문제가 있었습니다
MCU 가 PC 터미널 에게 다음 줄을 업로드를 요구하려면, 문자열에 문자 8개 들어와야한다는
전제가 있으나 만약 문자 7개만 들어온다면, MCU 가 무한정 대기에 들어가 문제를 발생시켰습니다

이 문제를 정정하기 위해, merge 기능을 만들기로 결정했습니다
문자열은 총 8개의 문자를 받기로 의도되어있는데, 총 7개의 문자만 들어온다면
그 다음에 들어오는 문자 1개를 이것과 합쳐 본래 의도의 형태로 만들었습니다

Merge 가 구현되기 위해서는, 현재 Stream의 상태가 어떤지 알아야,
그에 대응되는 대응을 할 수 있기에 상태를 먼저 정의했습니다

상태들은 다음으로 나눌 수 있음을 확인했습니다

STREAM_INUSE_NOERROR
: 가장 마지막으로 8개 입력이 들어왔거나, 0개 입력이 진행중일 때
STREAM_INUSE_ERROR_MERGEATTEMPT
: 가장 마지막으로 1~7개 입력이 들어왔을 때
STREAM_INUSE_ERROR_MERGEFAIL_FILLINGZERO
: 1~7개 입력 상태에서, 아무리 UART 입력을 재수신 하려고 해도 부족한 양이 안들어오는 경우
STREAM_INUSE_NOERROR_NOINPUT_FORLONG
: 0개 입력이 너무 오래동안 유지 된 경우

이런 상태들을 나누기 위해 enum 자료형 정의를 했고, 그 이유는 아래와 같습니다

- enum 방식을 포함해 상태를 나누는 방법은 다음이 있습니다

1. if { ... } else if { ... } else if.... else 방식
   : 복잡한 상태에 대해 가독성이 가장 떨어지는 방식

2. if (status = 1) 또는 if (status = 매크로)
   { if (작은 상태) { ... } } 방식
   : 이것도 충분히 이용할 가치가 있는 방식이며, 아래 기술할 enum 방식과 가장 닮았습니다
   하지만 이 매크로는 모든 변수에서 접근할 수 있는 만큼 매크로가 꼬일 위험성이 존재합니다

3. if (enum변수 = enum값) { if (작은 상태) { ... } } 방식
   : 오직 이 enum 변수에 대해서만 enum 값을 사용하는 만큼,
   이 enum 변수에만 종속된 매크로로 기능하기에 매크로가 꼬이지 않습니다
   따라서 이 구조가, 상태전이를 보여주기에 가장 적합합니다

기타 마주했던 다양한 제약조건

: STM32F103RB내에 플래시 공간의 부족

: Nucleo 보드의 3.3V 헤더핀 한 개에서 제공할 수 있는 전력값 한계

: W25Q64에서 MCU 로 1024byte 를 꺼내는 속도가 MCU 에서 ILI9341로 1024byte 를 업로드 하는 속도에 비해 압도적으로 느린 문제

깃허브 링크

https://github.com/leafeons2/ILI9341-STM32F103RB